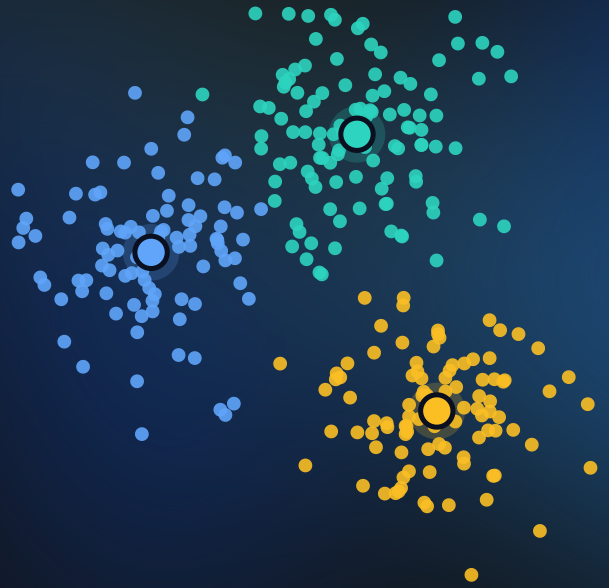
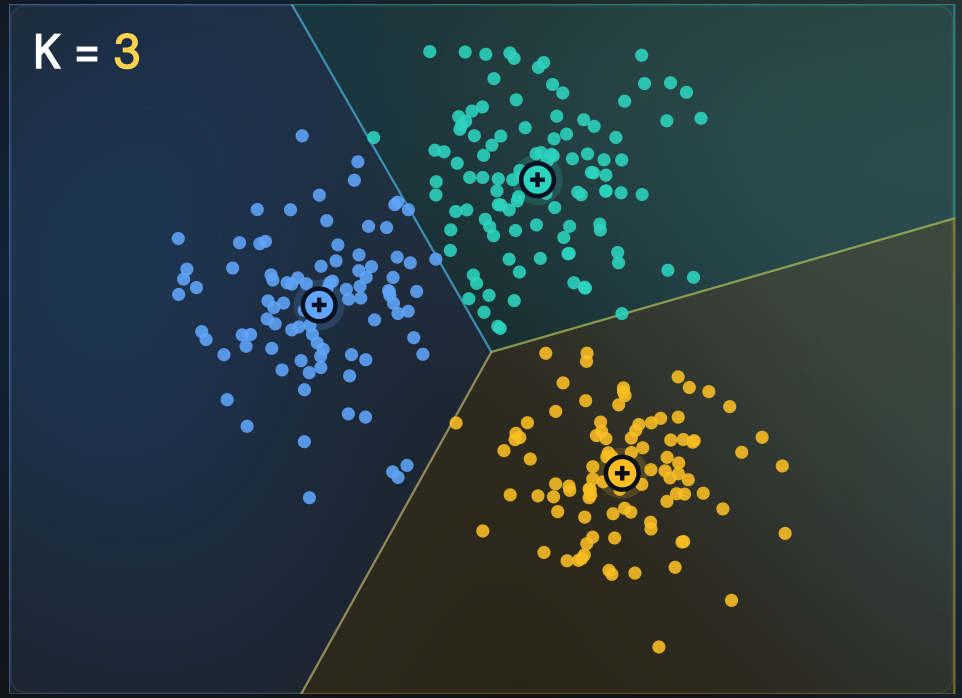
k-Means Clustering

Finding groups when there are no labels to check against



Unsupervised Learning · Foundations of Machine Learning

Same Points, Two Reasonable Answers



No y, no test accuracy

In supervised learning, held-out error tells us which model is better. Here there is nothing to compare against.

"Good" depends on the use

Two broad customer tiers or three targeted campaigns? Both splits are defensible. The data constrains the answer; it does not pick it.

So we need: **(1)** a precise definition of a good grouping, **(2)** an algorithm to find one, **(3)** tools to judge it without labels.

Where Clustering Shows Up

Customer segments
Group accounts by what they buy, then design one offer per group.

Image compression
Replace millions of colors with a palette of K representative ones.

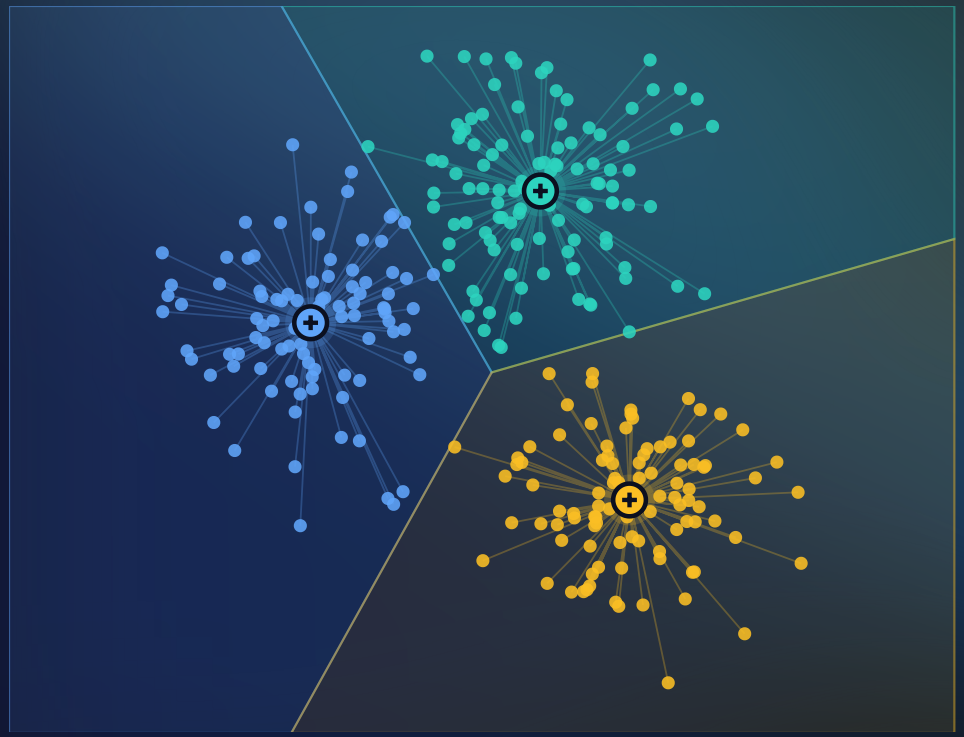
Facility placement
Put K depots, stores or cell towers close to the people they serve.

Grouping documents
Cluster text embeddings to find topics in support tickets or reviews.

Feature engineering
Use the cluster id, or distance to each centroid, as an input to a supervised model.

Exploration
A first look at a new dataset: what kinds of rows are in here?

The Idea: Summarize Each Group by a Center



K centroids

Pick K points $\mu_1, \ldots, \mu_K$ in feature space. Each is the "prototype" of one group.

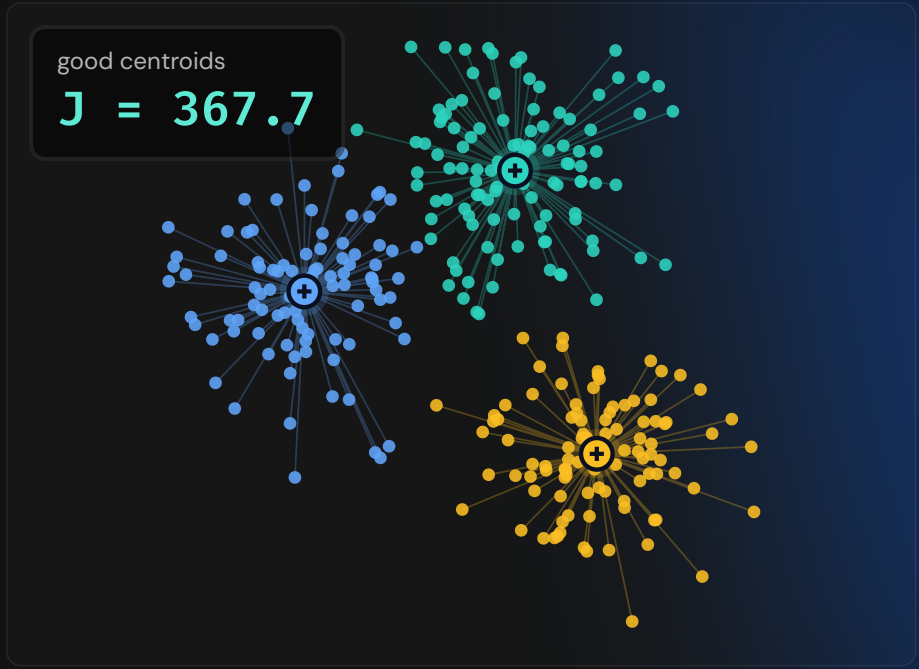
Nearest centroid wins

Every point belongs to the centroid it is closest to. The shaded cells show who owns which part of the plane.

Short lines = good clustering

A clustering is good when every point is close to its own centroid.

Scoring a Clustering: The Objective J



$$J = \sum_{i=1}^{n} \left\| x_i - \mu_{c_i} \right\|^2$$

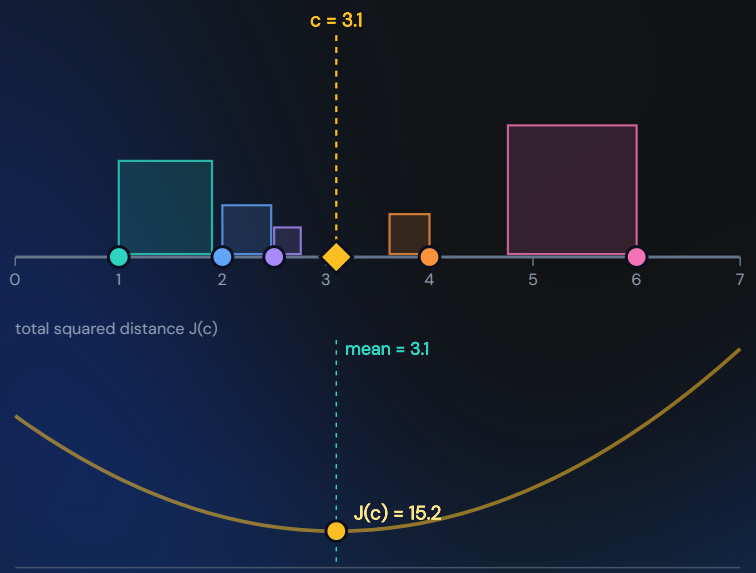
$c_i \in \{1, \ldots, K\}$: which cluster point i is in

μ_k: the centroid of cluster k

J is the total **squared length of all the lines**. Also called inertia or within-cluster sum of squares (WCSS).

Two kinds of unknowns: the **assignments** c_i (discrete) and the **centroids** μ_k (continuous). We must choose both.

Why the Centroid Is the Mean



Fix the members of one cluster. Where should its center c go? Each square's area is one point's squared distance; J(c) is their total.

$$\frac{dJ}{dc} = -2\sum_i (x_i - c) = 0 \;\Rightarrow\; c = \frac{1}{n}\sum_i x_i$$

Same argument in every coordinate, so in d dimensions the best center is the **mean vector** of the cluster. Hence the name.

Absolute distance instead of squared → the best center is the **median** (k-medians).

Lloyd's Algorithm: Alternate Two Easy Steps

0 · Initialize

Place K centroids, e.g. on K random data points.

1 · Assign (fix μ, choose c)

Each point joins its nearest centroid: $c_i = \text{argmin}_k \|x_i - \mu_k\|^2$.

2 · Update (fix c, choose μ)

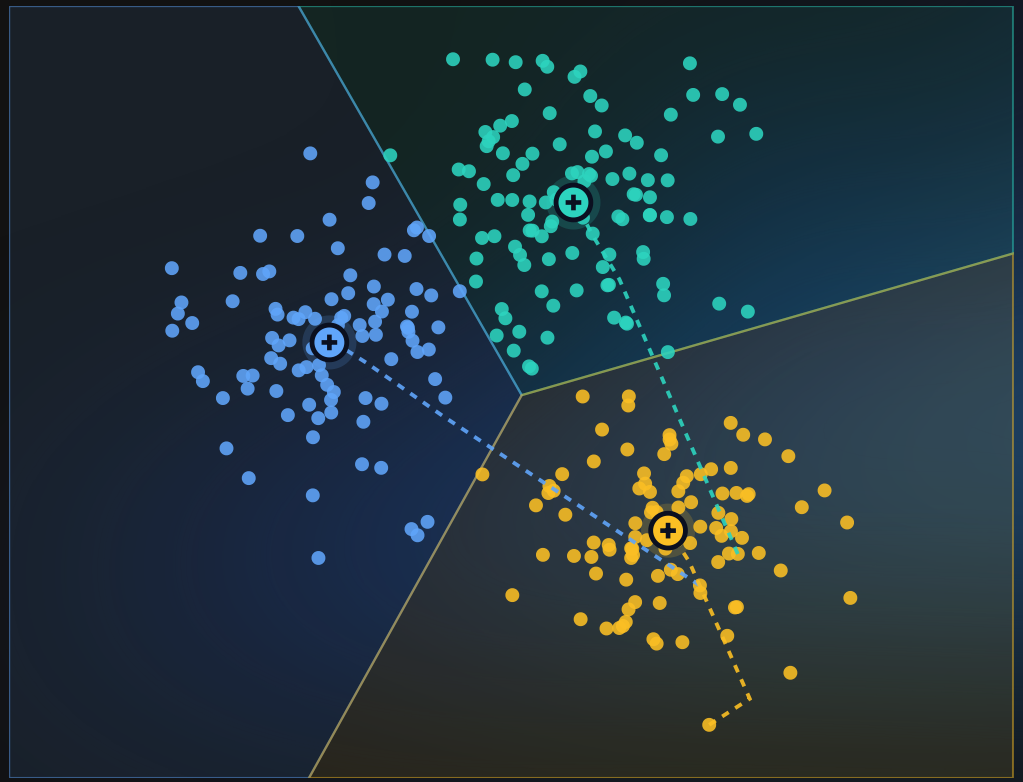
Each centroid moves to the mean of its points.

Repeat 1–2 until no point changes cluster

```python
import numpy as np

def kmeans(X, K, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    mu = X[rng.choice(len(X), K, replace=False)]
    for _ in range(iters):
        # assign: squared distance to each centroid
        d = ((X[:, None, :] - mu[None]) ** 2).sum(-1)
        c = d.argmin(axis=1)
        # update: mean of each cluster
        new = np.array([X[c == k].mean(0) for k in range(K)])
        if np.allclose(new, mu):
            break
        mu = new
    return c, mu
```

Watch It Run

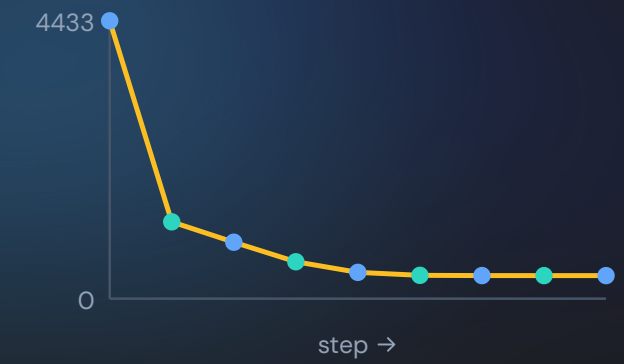


ITERATION 4

Converged

No point changed cluster. Done.

Objective J **367.7**



Worked Example: One Iteration by Hand



pt	x	$\|x - \mu_1\|$	$\|x - \mu_2\|$	cluster
A	(1, 1)	**0.00**	7.21	1
B	(1.5, 2)	**1.12**	6.10	1
C	(3, 3.5)	**3.20**	4.03	1
D	(5, 7)	7.21	**0.00**	2
E	(3.5, 5)	4.72	**2.50**	2
F	(4.5, 5)	5.32	**2.06**	2

centroids

μ_1 = (1.83, 2.17)

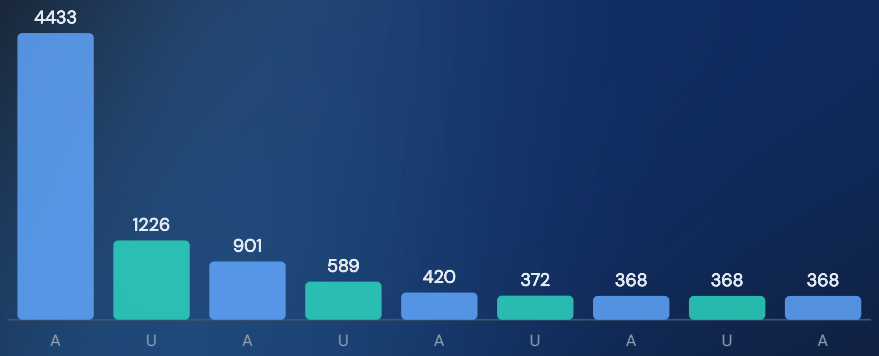
μ_2 = (4.33, 5.67)

objective J

22.00 → 9.17

reassign: no change → converged

Why Lloyd's Algorithm Always Stops



A = after assign · U = after update · the last A changes nothing: converged

Assign can't raise J

Each point switches only to a *closer* centroid.

Update can't raise J

The mean is the best center for a fixed group.

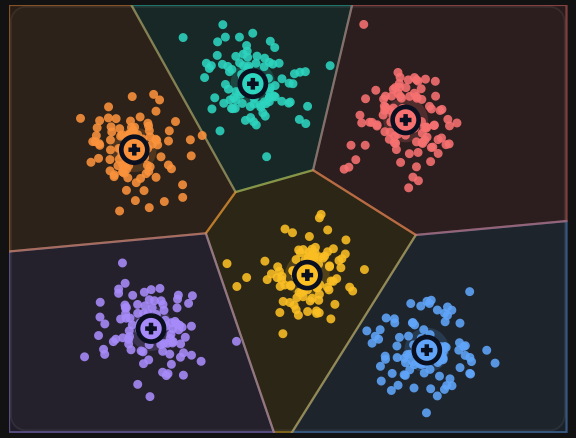
Finitely many groupings

J strictly drops until nothing changes, and no grouping can repeat, so the loop must end.

In practice: a handful of iterations. Typical cost per iteration: O(n·K·d).

But Only to a Local Minimum

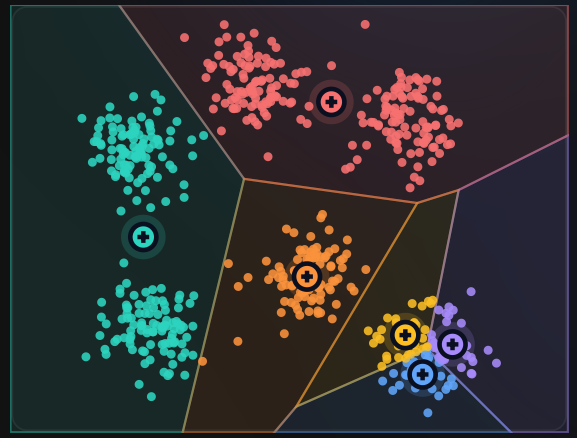
Six well-separated blobs, K = 6. Three random starts, each run until it stops:



Found the six groups J = 1,145



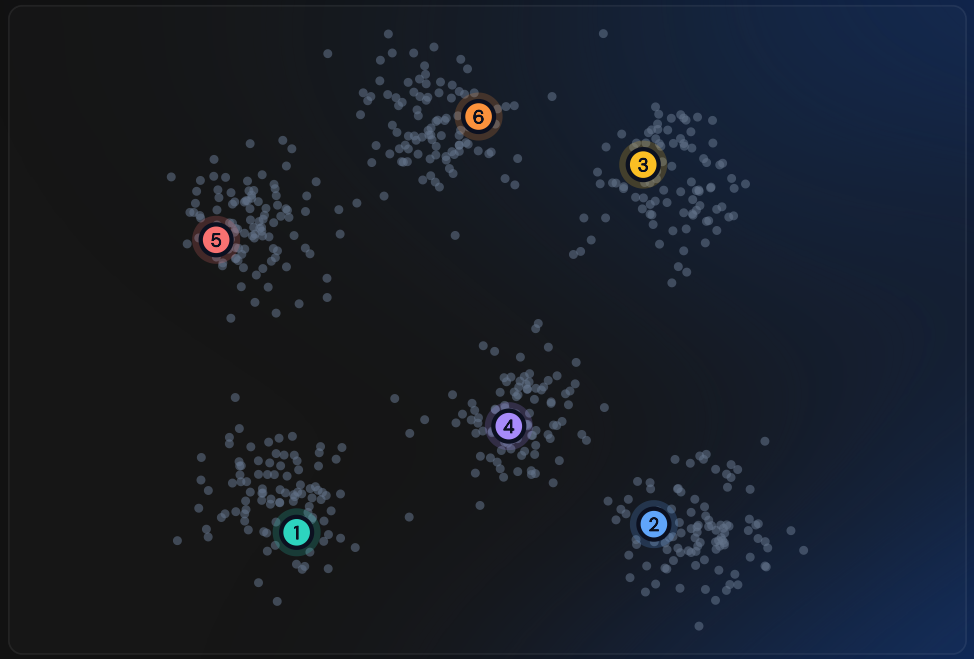
Stuck: 1 centroid, 2 blobs J = 2,473



Stuck: 3 centroids, 1 blob J = 5,496

Stuck runs are stable: moving any one centroid a little would make J worse, so Lloyd's algorithm never escapes. The result depends on where you start.

k-means++: Spread Out the Starting Centroids

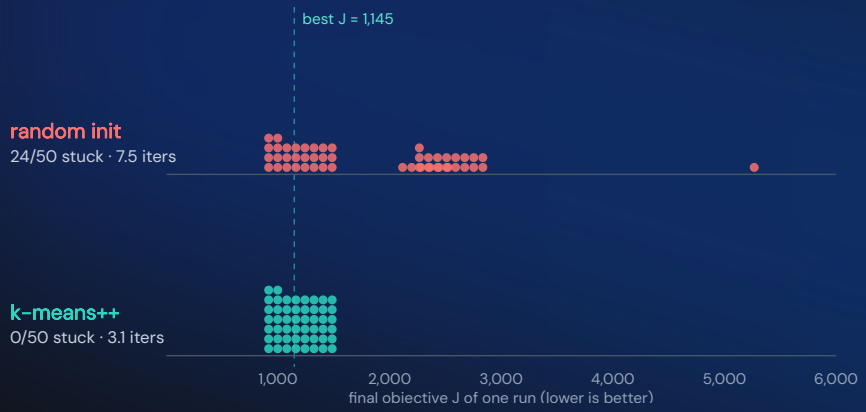


1. First centroid: a random data point.
2. Every point gets a weight $D(x)^2$: its squared distance to the *nearest* centroid chosen so far.
3. Pick the next centroid at random with probability $\propto D(x)^2$.
4. Repeat until there are K.

$$P(\text{pick } x) = \frac{D(x)^2}{\sum_{x'} D(x')^2}$$

Bigger, brighter amber dots = more likely to be picked next. The dashed ring marks the point that was actually drawn.

Random Starts vs. k-means++



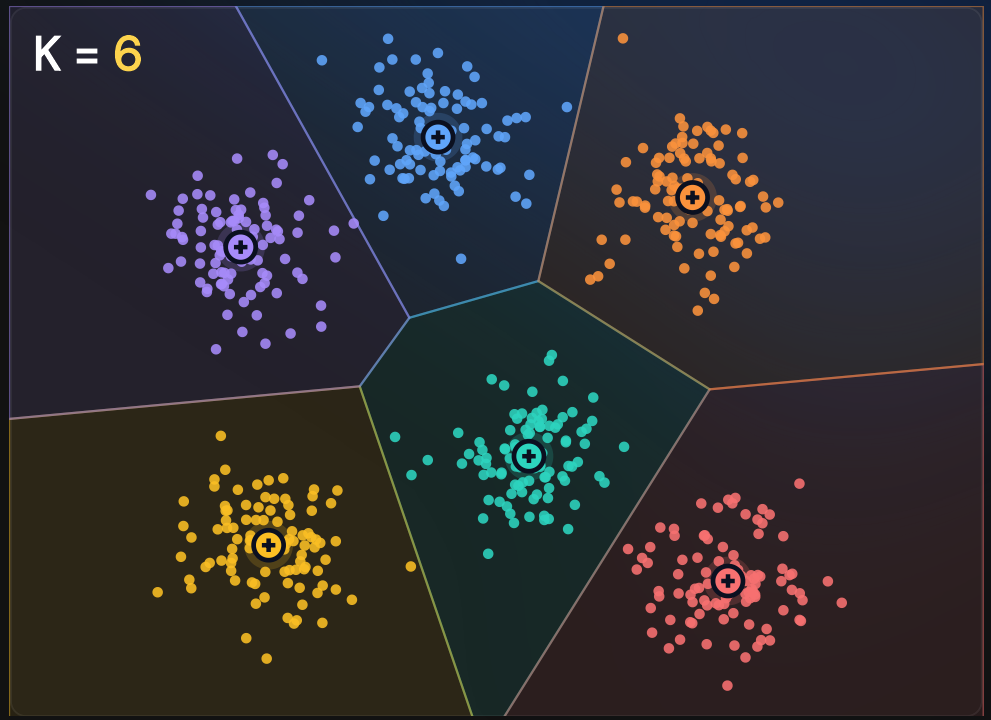
50 single runs each (n_init=1) on the six-blob data. Each dot is the final J of one run.

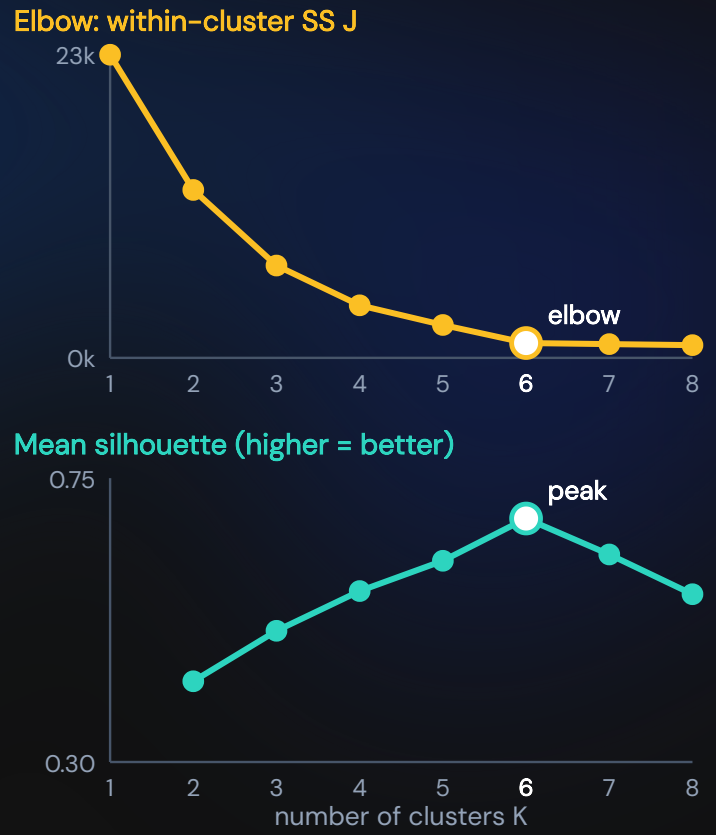
```
from sklearn.cluster import KMeans

km = KMeans(
    n_clusters=6,
    init="k-means++",  # the default
    n_init=10,         # 10 starts, keep the lowest J
    random_state=0,
).fit(X)
km.inertia_            # J of the best run
```

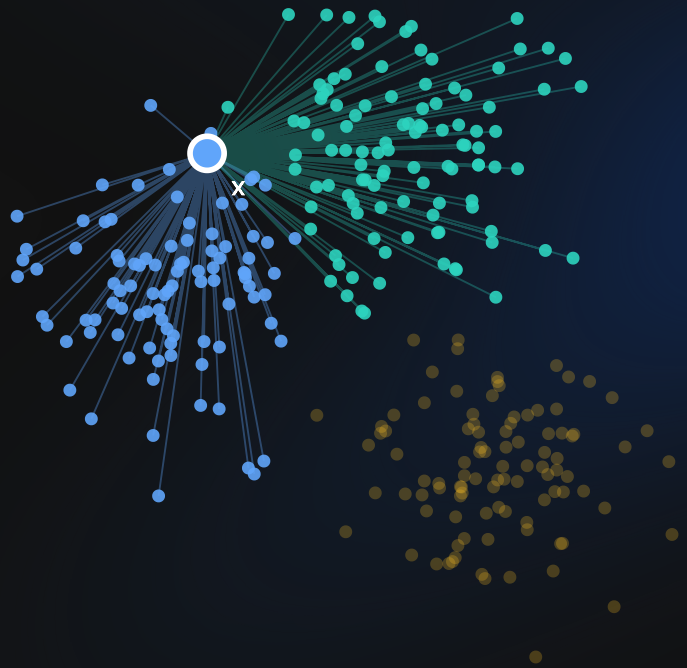
The default n_init="auto" runs once with k-means++ and 10 times with random starts.

Choosing K: The Elbow and the Silhouette





The Silhouette of One Point



a(x): average distance to the other points **in its own cluster**

`a = 1.78`

b(x): average distance to the points of the **nearest other** cluster

`b = 2.37`

s(x) = (b − a) / max(a, b)

`s = 0.25`

+1 deep inside its cluster · 0 on the border · negative: probably in the wrong cluster

Silhouette Plots: Every Point, Every Cluster



Choosing K in Practice

Elbow of J

Quick and cheap. J always falls with K; look for where it stops falling fast. Often there is no sharp elbow.

Silhouette

Mean score and the per-point plot. Favors compact, well-separated groups.

Stability

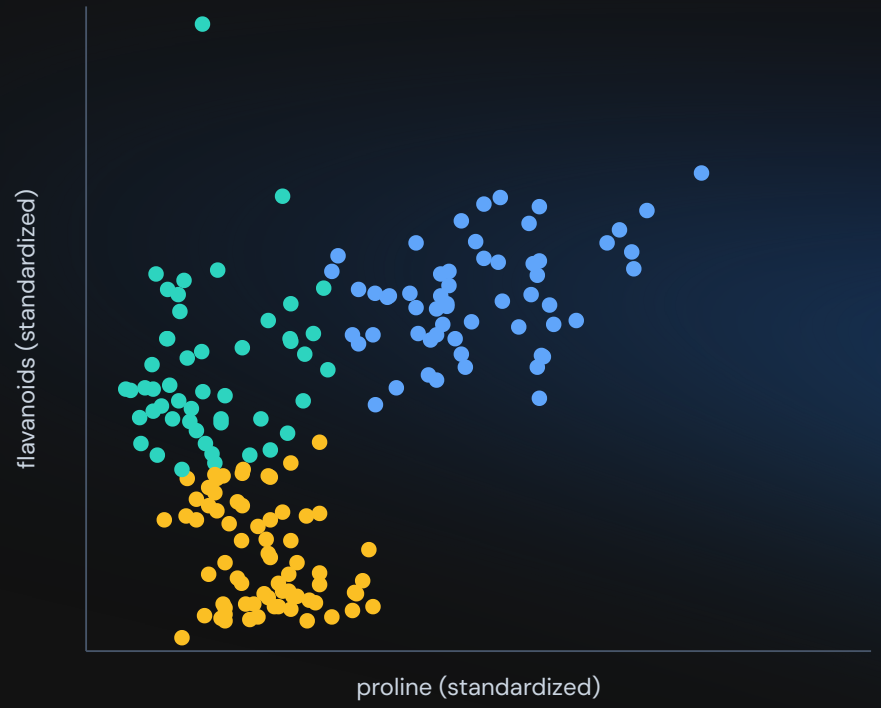
Refit with another seed or a bootstrap sample. Real structure comes back; noise moves around.

The use case

Can you name and act on each cluster? Three segments you can describe beat eight you can't.

No method finds the "true" K. Use several, and report which K you chose and why.

Distance Is the Model: Scale Your Features



agreement with grape variety (ARI): **0.60**

Raw units. Proline spans ~1,400 units, flavanoids ~5. Euclidean distance is almost all proline, so k-means just slices along the proline axis.

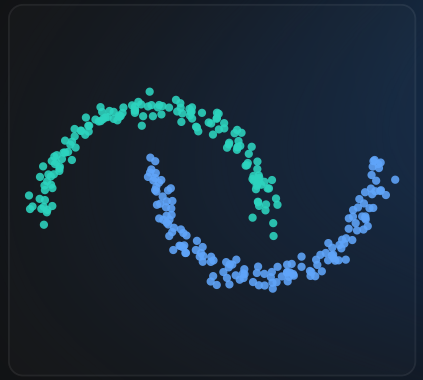
Standardized view, same clusters. The raw clusters are vertical bands: flavanoids was ignored.

Re-cluster after scaling. Both features count, and the clusters line up much better with the three grape varieties.

Where k-Means Fails

Curved shapes

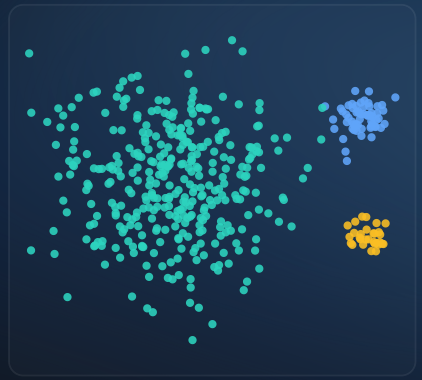
ARI 0.23



Clusters are not round blobs around a center

Unequal spread

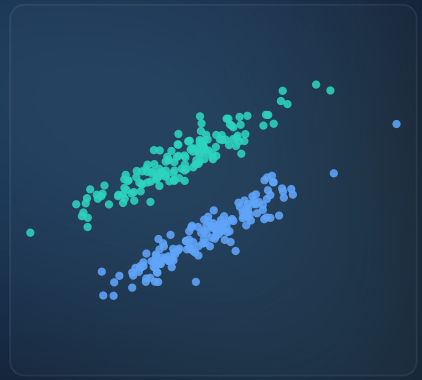
ARI 0.28



Splits the wide cluster, merges the two tight ones

Stretched

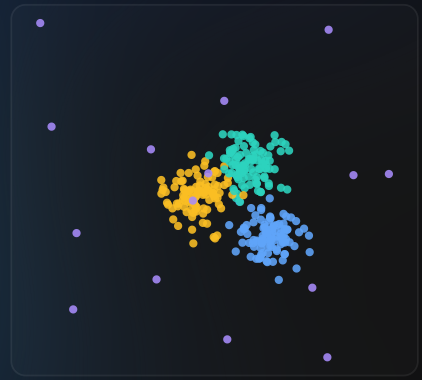
ARI 0.01



Distance is not equally meaningful in every direction

Outliers

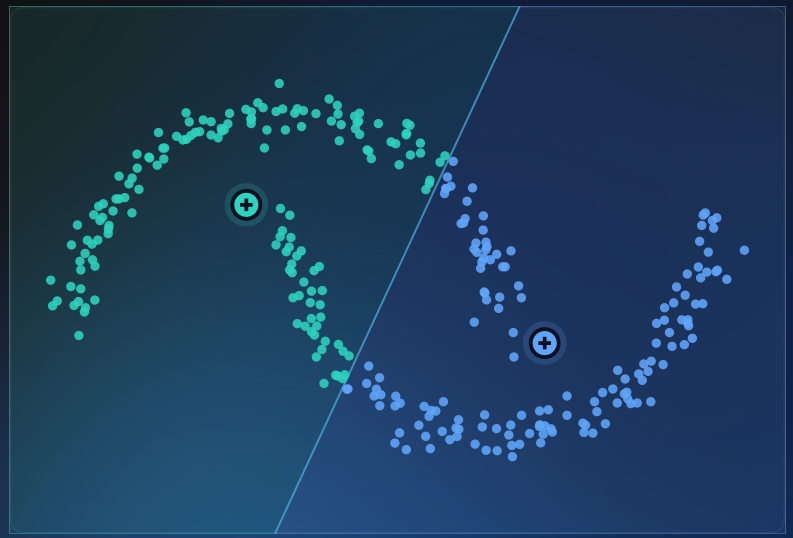
ARI 0.85



A few far points pull a centroid or take one over

The true groups: what k-means should have found.

Why: Every Border Is a Straight Line



Point x is closer to μ_1 than to μ_2 when

$$\|x-\mu_1\|^2 < \|x-\mu_2\|^2 \iff 2(\mu_2-\mu_1)^\top x < \|\mu_2\|^2 - \|\mu_1\|^2$$

The $\|x\|^2$ terms cancel, so the condition is **linear in x**: the border is the perpendicular bisector of μ_1 and μ_2.

Each cluster is an intersection of half-planes, so it is always a **convex** region. No setting of k-means can wrap around a moon.

Next deck: hierarchical and density-based clustering define clusters by connectivity instead, and can follow curved shapes.

k-Means on Real Data

Three datasets where k-means is a good fit

Customer segments
440 wholesale clients, 6 spending categories

Image colors
153,600 pixels in RGB space

Depot placement
20,640 California locations

k-Means in scikit-learn

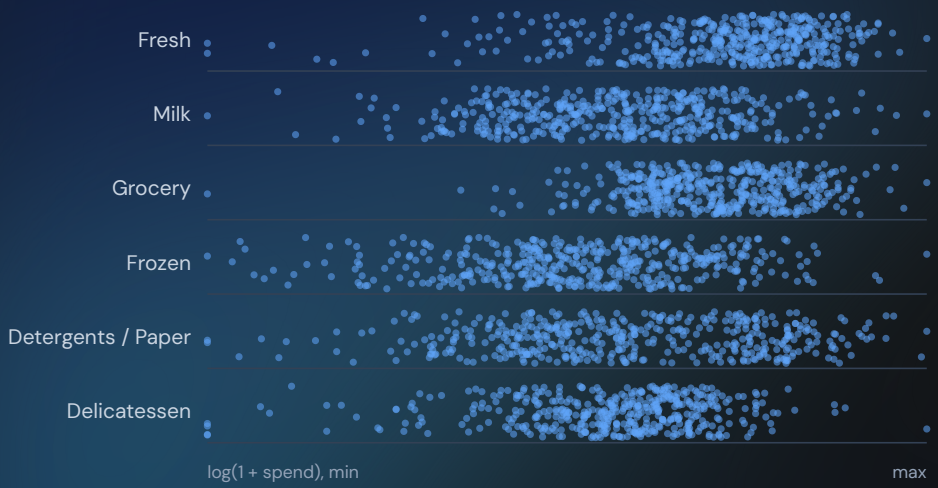
```
from sklearn.cluster import KMeans
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# X: (n_rows, n_features) numeric array or DataFrame, no labels
model = make_pipeline(
    StandardScaler(),
    KMeans(n_clusters=4, n_init=10, random_state=0),
)
labels = model.fit_predict(X)      # cluster id (0..3) for every row
km = model[-1]; km.inertia_        # J, measured on the scaled data
centers = model[0].inverse_transform(km.cluster_centers_)  # back to original units
model.predict(X_new)               # nearest centroid for new rows
```

Big data: `MiniBatchKMeans` updates centroids from small random batches. Much faster, slightly higher J.

Not a classifier: cluster ids are arbitrary names. Cluster 2 in one run may be cluster 0 in the next.

Example 1: Wholesale Customers

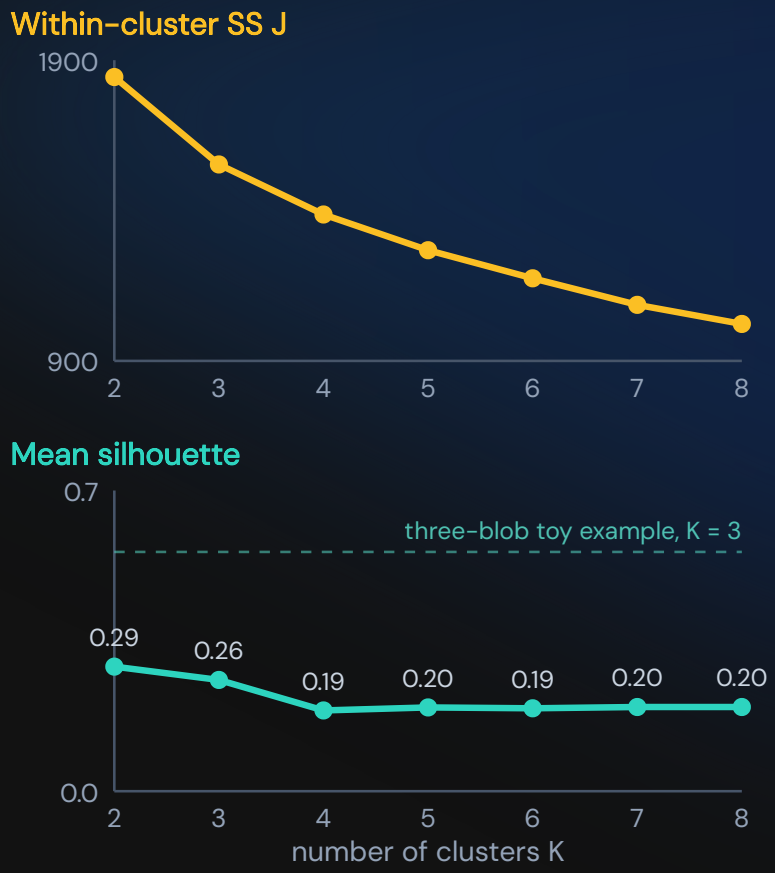


440 clients of a Portuguese food distributor. One row per client: **annual spend** on 6 product categories. Also a Channel column (hotel/restaurant/café vs. retail) that we **hide from k-means** and keep for checking.

Raw spend is heavily skewed. A few huge accounts would dominate every distance.

log(1 + spend), then standardize. Now "twice as much" counts the same at every scale.

Wholesale: How Many Segments?



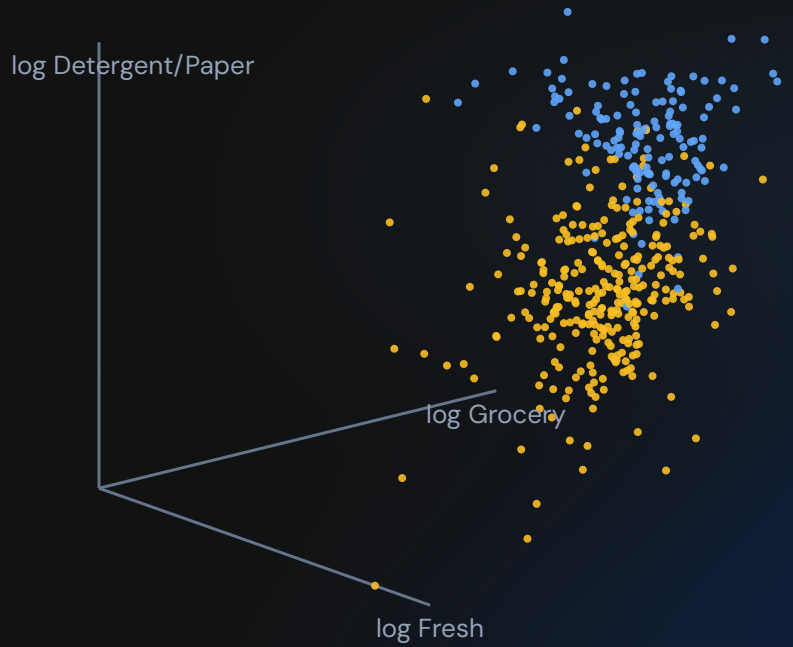

```python
import numpy as np, pandas as pd
from sklearn.preprocessing import FunctionTransformer

df = pd.read_csv("wholesale_customers.csv")
spend = df.drop(columns=["Channel", "Region"])
model = make_pipeline(
    FunctionTransformer(np.log1p), StandardScaler(),
    KMeans(n_clusters=4, n_init=10, random_state=0))
df["segment"] = model.fit_predict(spend)
```

No sharp elbow, and the silhouette peaks at **K = 2** (0.29), far below the toy data. Real customers form overlapping groups.

We pick **K = 4** for the marketing use, and check that it is **stable**: the median agreement across 20 single-start refits is ARI 0.83.

Wholesale: The Segments in 3D



3 of the 6 features, log scale. Drag to rotate, hover for values.

Colored by Channel, which k-means never saw: hotel/restaurant/café vs retail.

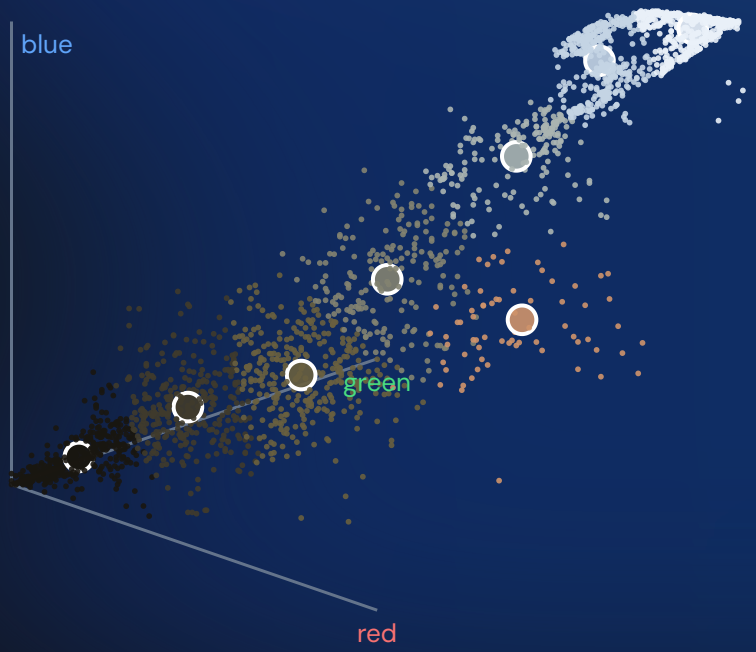
K = 2 recovers the channel split for **85.9%** of clients: 244 + 134 of 440, without ever seeing that column.

Wholesale: Naming the Segments

cluster	n	Fresh	Milk	Grocery	Frozen	Detergent/Paper	Deli	Hotel/Rest./Café · Retail
Small food-service accounts	132	7.8k	1.1k	1.8k	1.4k	224	435	131 · 1
Small grocery-type shops	61	1.1k	7.0k	10.5k	340	4.5k	295	21 · 40
Fresh-heavy restaurants	129	17.0k	3.1k	3.4k	4.3k	529	1.4k	122 · 7
Big grocery retailers	118	7.8k	8.5k	12.3k	1.3k	4.6k	1.7k	24 · 94

above-average spend (standardized log) · below-average spend · numbers = median annual spend (m.u.)

Example 2: Pixels Are Points in Color Space

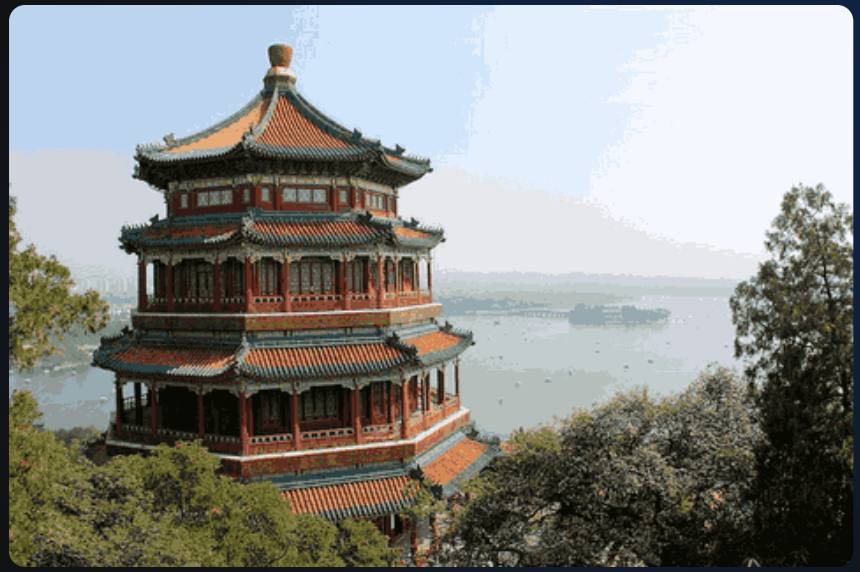


Every pixel is a 3D point: R, G, B. Here, 2,500 pixels of a photo, each drawn in its own color.

k-means with K = 8: the large spheres are the centroids. Each pixel is redrawn in its centroid's color.

Why it fits: all three features share one unit (0–255), so no scaling, and squared color error is exactly what we want to minimize.

Color Quantization: K Colors for 153,600 Pixels



K = 32

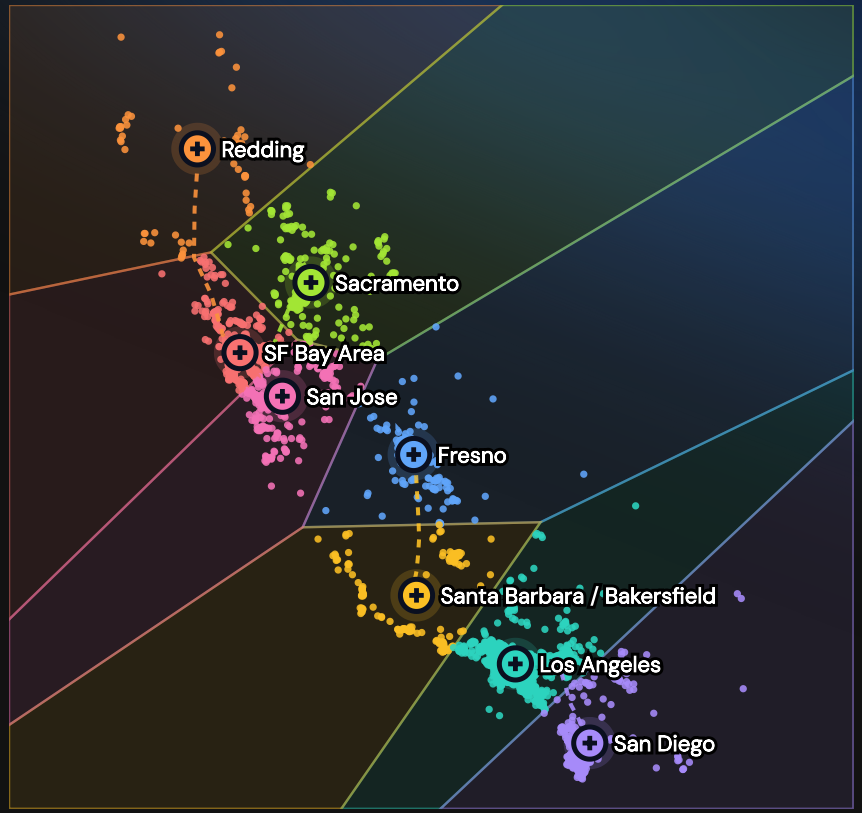
32 colors, each one a centroid in RGB space



bits per pixel	avg squared color error ($\times 10^{-4}$)
5	28.6

```
from sklearn.utils import shuffle
img = load_sample_image("china.jpg") / 255          # (427, 640, 3)
pixels = img.reshape(-1, 3)                         # one row per pixel
sample = shuffle(pixels, random_state=0, n_samples=5000)
km = KMeans(n_clusters=16, n_init=4, random_state=0).fit(sample)
quantized = km.cluster_centers_[km.predict(pixels)].reshape(img.shape)
```

Example 3: Where to Put 8 Depots?



CONVERGED

mean squared distance to the nearest depot

2,878 km²

On average a block group is **43 km** from its depot. The busiest depot serves **8,755** of 20,640 block groups.

When Is k-Means a Good Choice?

Works well when…

- Features are **numeric** and Euclidean distance is meaningful (after scaling)
- Groups are **compact and roughly round**, of similar size and spread
- You want **prototypes**: a palette, depot sites, a typical customer
- Data is **large**: millions of rows, O(n·K·d) per iteration

Reach for something else when…

- Clusters are **curved, nested or stretched** → DBSCAN, hierarchical (next deck), GMM
- Densities or sizes **differ a lot** → HDBSCAN, GMM
- **Outliers** matter → k-medoids, DBSCAN, or remove them first
- Features are **categorical** → k-modes, or a different distance

Summary

Objective

Minimize $J = \Sigma \|x_i - \mu_{c_i}\|^2$. Centroids are means; borders are straight lines.

Algorithm

Lloyd: assign, update, repeat. J never rises, but only reaches a local minimum.

Initialization

k-means++ seeding plus n_init restarts. Fix random_state.

Choosing K

Elbow, silhouette, stability, and the use case. No true K.

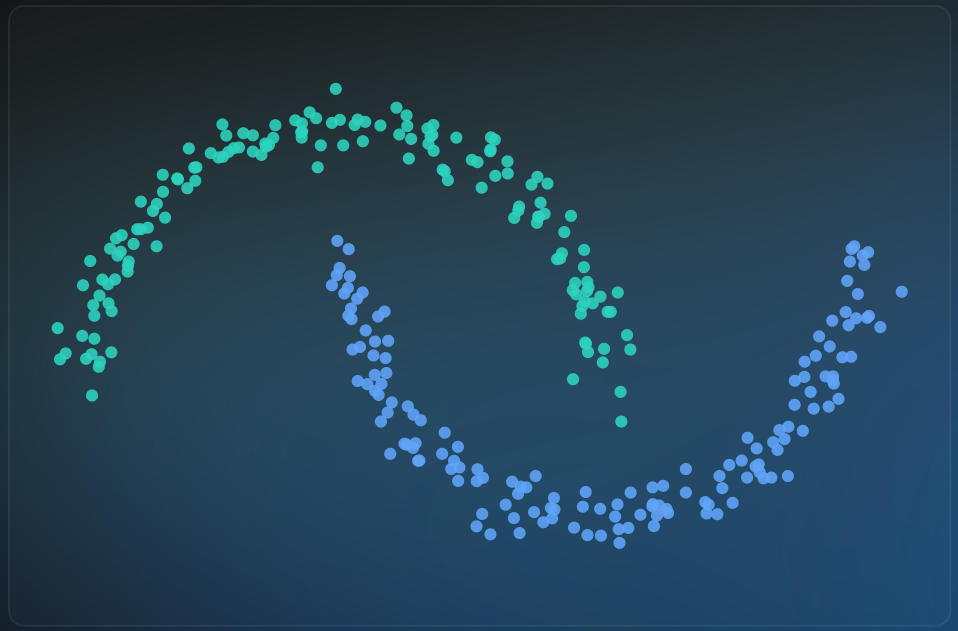
Preprocessing

Distance is the model: log skewed features, then standardize.

Limits

Convex, similar-size groups only; sensitive to outliers.

Next: Clusters Without Centroids



k-means: a cluster is the region around a center. Straight borders cut the moons in half.

Next deck: a cluster is a set of points **connected** through dense neighborhoods. **Hierarchical clustering**, **DBSCAN** and **HDBSCAN** can follow shapes like these and label outliers as noise.